APM Deployment & Distributed Tracing

Step-by-step lab manual for your existing Ubuntu ELK environment

Four VMs: esnode-1, esnode-2, esnode-3, and a separate Kibana VM. Includes Fleet-managed APM, a two-service Python application, sampling, errors, trace/log correlation, Dev Tools verification, troubleshooting, and lab assessments.

LAB TRACK	OUTCOME
L1 | Fleet & APM Server	APM intake is healthy and reachable
L2 | Instrument services	order-service and payment-service appear in APM
L3 | Tracing & failures	Inspect normal, slow and failed distributed traces
L4 | Sampling & naming	Filter by service/version/environment and compare rates
L5 | Log correlation	Use trace.id to join app logs with traces
Q | Verify & troubleshoot	Validate Elasticsearch data streams and fix failures

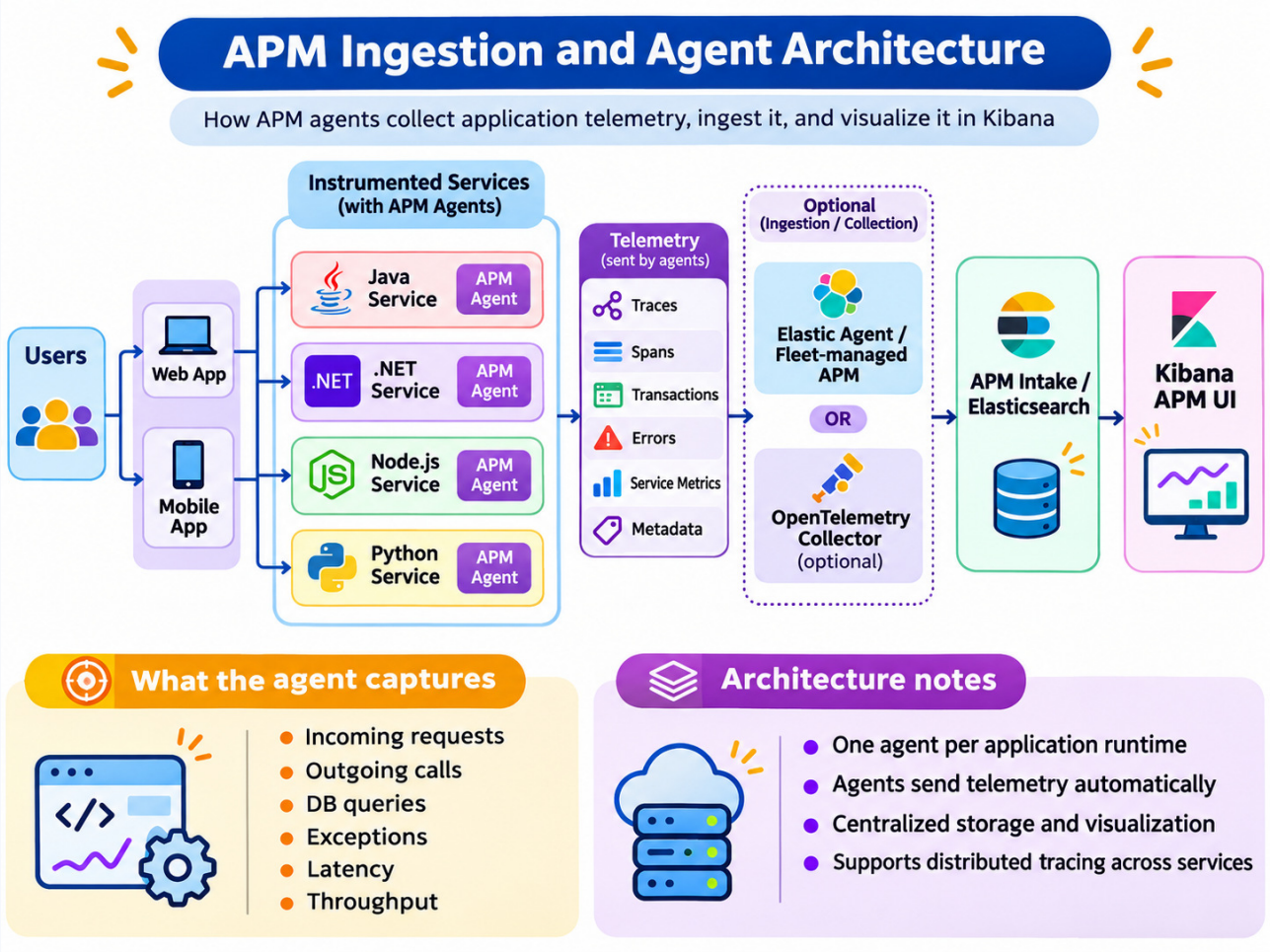
Lab safety: The instructions use a private/isolated VM network. Plain HTTP on port 8200 is for training only; use HTTPS and valid CA trust for production. Replace example IP addresses and never publish passwords or APM tokens.

Prerequisite: Elasticsearch and Kibana are already working. This manual does not reinstall your cluster.

Prepared against Elastic documentation available on 05 November 2026. Menu labels vary by Stack version; use Kibana global search when needed.

1 | Topology and learning objectives

Use Fleet-managed APM Server on an already-enrolled Elastic Agent attached to the Kibana VM (or a separate capable VM). For lab convenience, run two Python Flask services on esnode-1 if it has spare resources; a separate application VM is better for production.



Conceptual APM overview (illustration). The actual path here is Python APM agents → Fleet-managed APM Server → Elasticsearch → Kibana.

VM / role	What runs there
esnode-1	Elasticsearch node; optional lab-only Python apps + Filebeat
esnode-2 / esnode-3	Elasticsearch members; no application APM agent required
Kibana VM	Kibana; existing Elastic Agent with Fleet Server and optionally APM integration
Fleet-managed APM	APM Server subprocess listens on 8200; routes telemetry to ES

2 | Network, ports and prerequisites

Port	Source → target	Use
5601	Browser → Kibana VM	Kibana UI
8220	Managed Elastic Agent → Fleet Server	Fleet control plane (HTTPS)
8200	Instrumented app → APM Server	APM intake (HTTP lab / HTTPS secure)
9200	APM Server → Elasticsearch	Store and search telemetry (HTTPS)
5050 / 5051	Local sample Flask services	Order and payment requests
5044	Filebeat → Logstash (optional)	Log forwarding via existing pipeline

Check on each VM

```
hostname -f
ip -br a
cat /etc/os-release | head -5
free -h
df -h /
```

Check versions and health

On esnode-1 run (adjust the CA path if your installation differs):

```
sudo systemctl is-active elasticsearch
sudo ls /etc/elasticsearch/certs/http_ca.crt
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u elastic \
  'https://127.0.0.1:9200/_cluster/health?pretty'
```

On the Kibana VM:

```
sudo systemctl is-active kibana
sudo systemctl status elastic-agent --no-pager
sudo ss -lntp | grep -E ':5601|:8220|:8200'
```

Expected: Elasticsearch health is green or yellow with three intended nodes visible; Kibana works; a Fleet-managed Elastic Agent shows Healthy in Fleet. A red cluster or Kibana connection problem must be fixed first.

3 | L1 - Fleet-managed APM intake

3.1 Confirm existing Fleet setup

- Open Kibana → Management → Fleet → Agents (or global search for Fleet).
- Verify there is an enrolled Elastic Agent with status Healthy; check which VM it is running on and which Agent Policy it uses.
- If a Fleet Server already exists, do not install another Fleet Server. You can add Elastic APM to the policy of an existing Elastic Agent.
- If there is no Fleet Server, choose Add Fleet Server in Fleet and follow the exact install command generated by your version of Kibana. Quick Start is suitable only for an isolated training lab.
- Do not install a second Elastic Agent on the same host; Fleet Server and APM can run under one Elastic Agent with a policy containing both integrations.

3.2 Add the Elastic APM integration

- In Kibana, navigate to Integrations, find Elastic APM, and choose Add Elastic APM.
- Choose the existing policy attached to the Elastic Agent on your designated APM host (for example, Kibana VM).
- Set the APM intake binding to 0.0.0.0:8200 for a lab VM with network access, or restrict it to the host IP if the integration UI supports that. Do not confuse APM port 8200 with Fleet Server port 8220.
- Under Agent authorization, create a long, random secret token. Save it securely for the applications.
- Keep the Elasticsearch output and CA/TLS settings from the existing working Fleet policy. Save the integration and wait for Fleet policy application.
- Verify the Agent remains Healthy and inspect the policy for Elastic APM.

> The Kibana host does not need a standalone apm-server package when running Fleet-managed APM. The APM Server process is managed by Elastic Agent. If the integration is assigned to an empty policy, nothing listens on 8200 until a healthy agent is attached.

3.3 Verify intake listener

Run on the VM hosting the APM integration:

```
sudo ss -lntp | grep ':8200'
sudo systemctl status elastic-agent --no-pager
sudo journalctl -u elastic-agent -n 80 --no-pager
curl -i --max-time 5 http://127.0.0.1:8200/
```

Expected: a process is listening on 8200. An HTTP response proves reachability, but it is not evidence that data reached Elasticsearch. The next labs verify ingestion end-to-end.

4 | L2 - Install a two-service sample application

Run the app on esnode-1 for this training lab if resources permit; otherwise use another Ubuntu application VM. The services communicate with each other on localhost, so they must be started on the same machine for these scripts.

4.1 Install Python tools and create a lab folder

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl
mkdir -p ~/apm-lab
cd ~/apm-lab
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install flask 'elastic-apm[flask]' requests ecs-logging
```

4.2 Create writable application-log directory

```
sudo mkdir -p /var/log/apm-lab
sudo chown "$USER":"$(id -gn)" /var/log/apm-lab
sudo chmod 755 /var/log/apm-lab
```

4.3 Create a protected APM environment file

Copy the supplied apm.env.template to the lab folder and rename it to apm.env. Or create it manually:

```
cat > ~/apm-lab/apm.env <<'EOF'
ELASTIC_APM_SERVER_URL="http://REPLACE_APM_VM_IP:8200"
ELASTIC_APM_SECRET_TOKEN="REPLACE_WITH_SECRET_TOKEN"
ELASTIC_APM_TRANSACTION_SAMPLE_RATE=1.0
ELASTIC_APM_CENTRAL_CONFIG=false
EOF
chmod 600 ~/apm-lab/apm.env
```

- Replace REPLACE_APM_VM_IP with the VM that actually hosts the APM integration, and replace the token with the one set in Kibana.
- If APM intake uses HTTPS, use https:// plus ELASTIC_APM_SERVER_CA_CERT_FILE pointing to the trusted APM Server CA certificate. Do not reuse the Elasticsearch CA unless it actually signs the APM listener certificate.

5 | L2 - The instrumented services

These two files are fully included in the ZIP alongside the manual. They use the Elastic APM Flask integration and ECS JSON logging. The two Python processes share a distributed trace when the Order Service calls the Payment Service.

5.1 Create orders.py

```
cat > ~/apm-lab/orders.py <<'PY'
import logging
import os

from flask import Flask, jsonify, request
from elasticapm.contrib.flask import ElasticAPM

app = Flask(__name__)
apm = ElasticAPM(app)

# Import the HTTP library after APM is initialized.
import requests
import ecs_logging

log = logging.getLogger("order-service")
log.setLevel(logging.INFO)
handler = logging.FileHandler("/var/log/apm-lab/orders.json")
handler.setFormatter(ecs_logging.StdlibFormatter())
log.addHandler(handler)

@app.get("/health")
def health():
    return jsonify(status="healthy", service="order-service")

@app.get("/purchase")
def purchase():
    mode = request.args.get("mode", "normal")
    log.info("Purchase requested: mode=%s", mode)
    response = requests.get(
        "http://127.0.0.1:5051/charge",
        params={"mode": mode},
        timeout=6,
    )
    if not response.ok:
        log.error("Payment failed: status=%s", response.status_code)
        raise RuntimeError("Payment service returned an error")
    log.info("Purchase completed")
    return jsonify(status="ok", payment=response.json())

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5050, debug=False, use_reloader=False)
```

```
PY
```

6 | L2 - Payment service and startup

6.1 Create payments.py

```
cat > ~/apm-lab/payments.py <<'PY'
import logging
import time

import ecs_logging
import elasticapm
from elasticapm.contrib.flask import ElasticAPM
from flask import Flask, jsonify, request

app = Flask(__name__)
apm = ElasticAPM(app)

log = logging.getLogger("payment-service")
log.setLevel(logging.INFO)
handler = logging.FileHandler("/var/log/apm-lab/payments.json")
handler.setFormatter(ecs_logging.StdlibFormatter())
log.addHandler(handler)

@app.get("/health")
def health():
    return jsonify(status="healthy", service="payment-service")

@app.get("/charge")
def charge():
    mode = request.args.get("mode", "normal")
    log.info("Charging request: mode=%s", mode)
    with elasticapm.capture_span(
        "validate-inventory", span_type="custom",
        span_subtype="inventory", span_action="validate"
    ):
        time.sleep(1.5 if mode == "slow" else 0.04)
    if mode == "fail":
        log.error("Simulated payment failure")
        raise RuntimeError("Simulated payment failure for training")
    log.info("Charge completed")
    return jsonify(status="accepted", amount=25)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5051, debug=False, use_reloader=False)
PY
```

6.2 Start Payment in terminal A

```
cd ~/apm-lab
source .venv/bin/activate
set -a; source ./apm.env; set +a
export ELASTIC_APM_SERVICE_NAME=payment-service
export ELASTIC_APM_ENVIRONMENT=lab
export ELASTIC_APM_SERVICE_VERSION=1.0.0
python payments.py
```

Expected: Flask runs locally on http://127.0.0.1:5051. Leave this terminal open.

6.3 Start Order in terminal B

```
cd ~/apm-lab
source .venv/bin/activate
set -a; source ./apm.env; set +a
export ELASTIC_APM_SERVICE_NAME=order-service
export ELASTIC_APM_ENVIRONMENT=lab
export ELASTIC_APM_SERVICE_VERSION=1.0.0
python orders.py
```

Expected: Flask runs locally on http://127.0.0.1:5050. Do not use flask --debug or its reloader: the APM agent normally avoids sending events in debug mode.

7 | L3 - Generate normal, slow and failed traces

7.1 Verify local endpoints in terminal C

```
curl -s http://127.0.0.1:5050/health
curl -s http://127.0.0.1:5051/health
curl -s http://127.0.0.1:5050/purchase?mode=normal
```

Expected: healthy JSON for both /health endpoints; a successful purchase response. Payment is invoked through an instrumented HTTP request from Order.

7.2 Normal load - baseline

```
for i in $(seq 1 30); do
  curl -s -o /dev/null -w '%{http_code} ' \
    'http://127.0.0.1:5050/purchase?mode=normal'
done
echo
```

Expected: mostly 200 responses. APM shows the Order transaction and a Payment transaction connected in one trace; HTTP spans and a custom validate-inventory span may be present.

7.3 Intentionally slow requests

```
for i in $(seq 1 7); do
  curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    'http://127.0.0.1:5050/purchase?mode=slow'
done
```

Expected: 200 responses with noticeably longer latency (roughly 1.5 seconds plus overhead).

7.4 Intentionally failed requests

```
for i in $(seq 1 5); do
  curl -s -o /dev/null -w '%{http_code}\n' \
    'http://127.0.0.1:5050/purchase?mode=fail'
done
```

Expected: 500 responses. Both services may report errors; the Payment Service raises the first simulated exception. Do not send actual customer/payment data.

8 | L3 - Analyze traces in Kibana

8.1 Application service inventory

- Go to Kibana → Observability → Applications → Service Inventory (or use global search).
- Set time range to Last 30 minutes, environment to lab.
- Find order-service and payment-service. Verify throughput, response times, and error rates.
- Open order-service → Transactions → the GET /purchase transaction (the exact transaction name may vary).
- Open a trace sample or trace waterfall. Locate the outbound HTTP span and the downstream payment-service transaction.
- Compare the normal and slow traces, then inspect errors from the simulated failures.

8.2 Expected observability evidence

View	What to observe
Service Inventory	Two named services: order-service, payment-service
Transactions	GET /purchase, durations, throughput
Trace waterfall	Order → HTTP request → Payment; nested custom span
Latency	Slow trace noticeably longer than normal trace
Errors	RuntimeError with stack trace for failure case

8.3 Screenshots for learner evidence

- ☐ Fleet → Agent and Elastic APM integration Healthy
- ☐ APM Service Inventory with both services and environment lab
- ☐ Normal trace waterfall (service boundaries visible)
- ☐ Slow trace waterfall (1.5-second custom span visible)
- ☐ APM Errors panel containing simulated RuntimeError

9 | Query APM data directly in Dev Tools

Open Kibana → Dev Tools → Console. These queries use Elastic APM data streams, not your syslog-v1 or other lab indices.

9.1 List available streams

```
GET _data_stream/traces-apm*
GET _data_stream/logs-apm.error*
GET _data_stream/metrics-apm*
```

Expected: trace, error and metrics streams for the namespace configured in the APM integration, typically default. Metrics may take longer to appear.

9.2 Confirm indexed transactions/spans

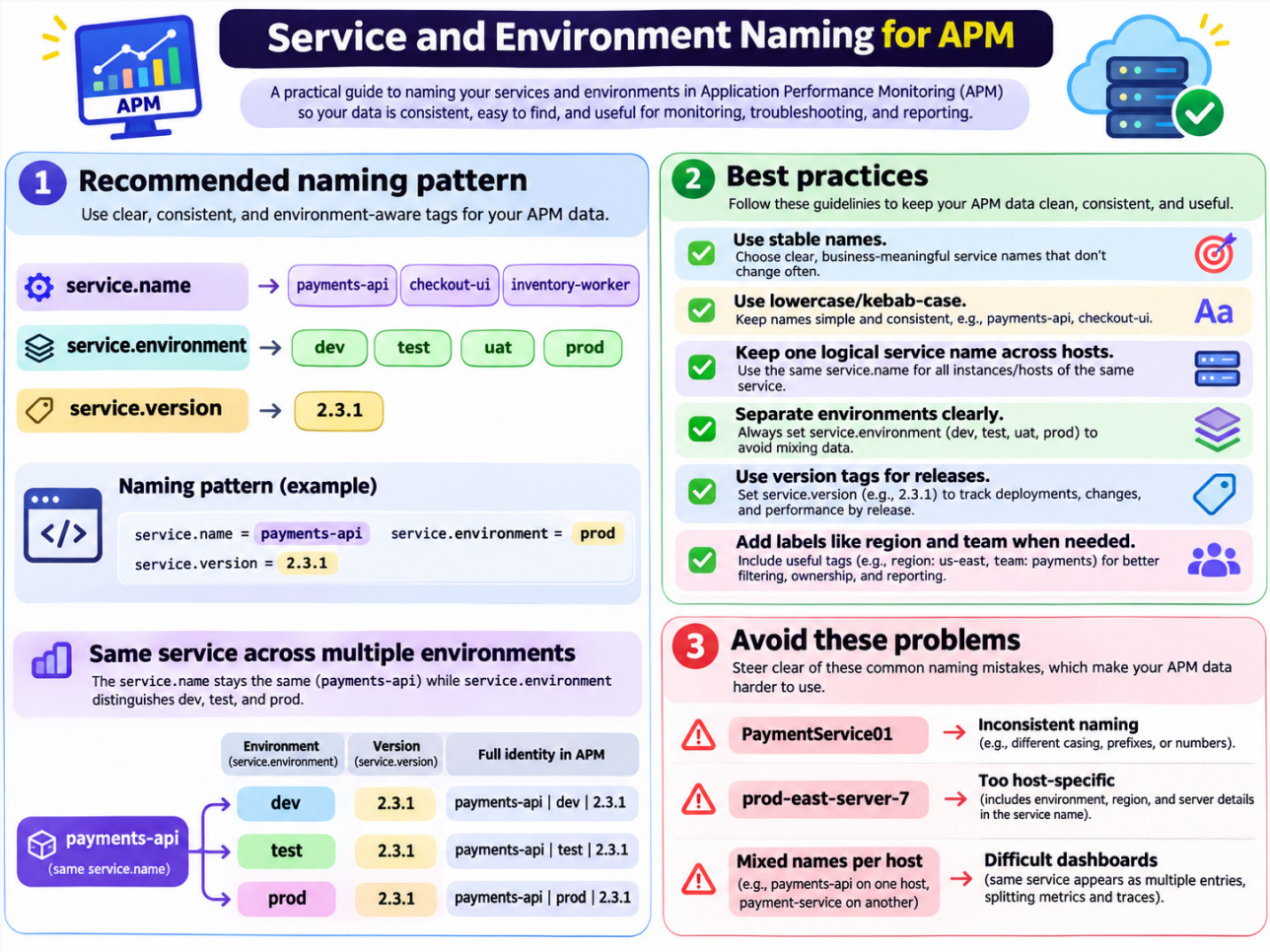
```
GET traces-apm*/_search
{
  "size": 5,
  "sort": [{"@timestamp": "desc"}],
  "_source": [
    "@timestamp", "service.name", "service.environment",
    "trace.id", "transaction.id", "span.id",
    "transaction.name", "span.name", "event.outcome"
  ]
}
```

9.3 Filter your application and environment

```
GET traces-apm*/_search
{
  "size": 10,
  "query": { "bool": { "filter": [
    { "term": { "service.name": "order-service" }},
    { "term": { "service.environment": "lab" }}
  ] } }
}
```

Expected: trace documents with a matching service name, environment, and a nonempty trace.id. If 0 hits, verify time, policy namespace, intake connectivity and agent warnings.

10 | L4 - Service/environment naming and sampling



Service naming concept. For the lab use order-service and payment-service, environment lab, version 1.0.0.

10.1 Filter and compare

- In APM Service Inventory, filter by lab.
- Restart only the Order Service with ELASTIC_APM_SERVICE_VERSION=1.1.0, send 10 requests, and inspect deployment/version grouping where available.
- Return the version to 1.0.0 for a clean class baseline.

10 | L4 - Sampling exercise (continued)

10.2 Run 10% head-based sampling

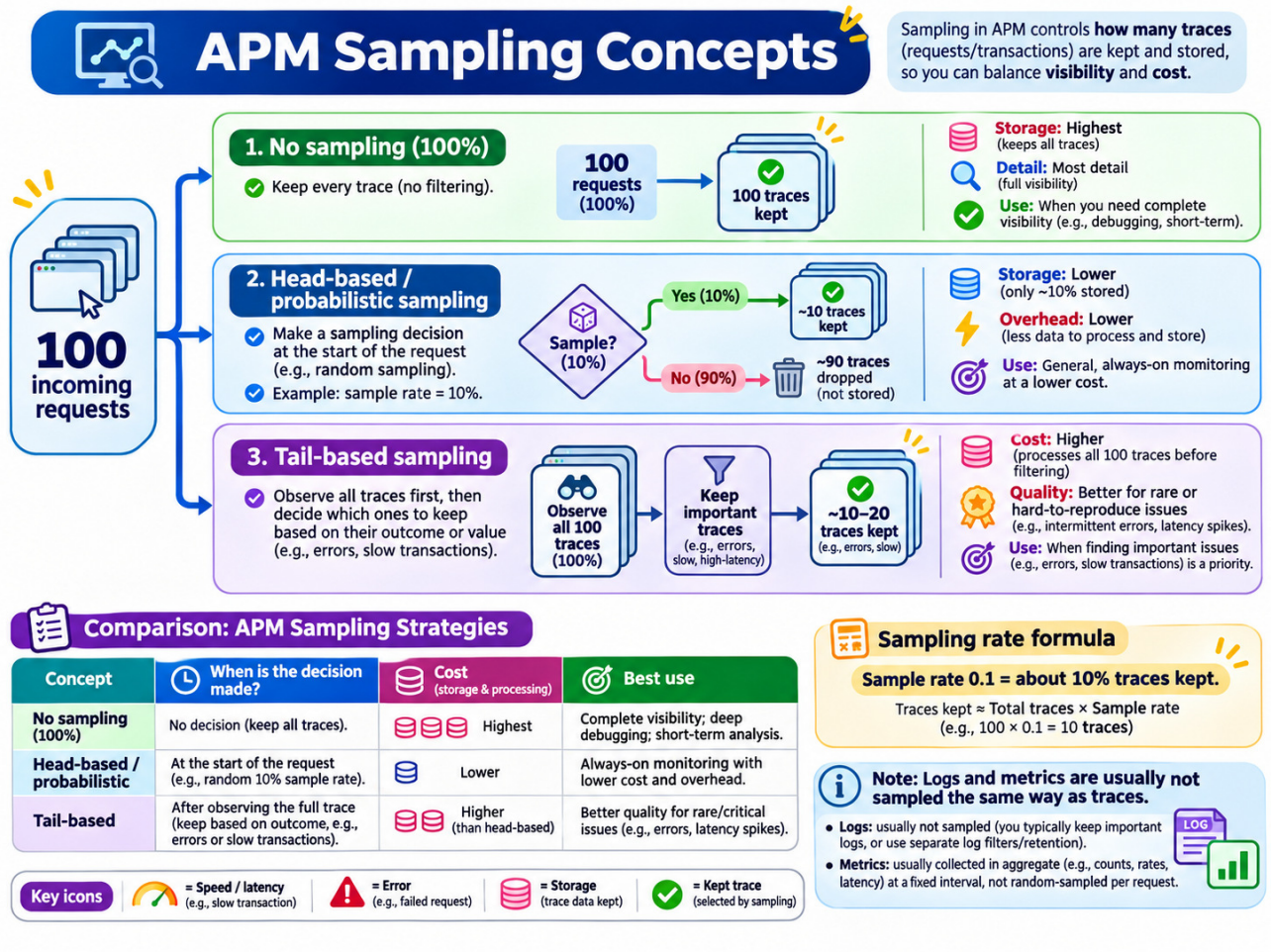
Stop both app processes (Ctrl+C), then change the rate in ~/apm-lab/apm.env to 0.1. Restart both using the commands in Lab 2. Keep central config disabled for this exercise, as central settings may override local sampling.

```
sed -i 's/TRANSACTION_SAMPLE_RATE=1.0/TRANSACTION_SAMPLE_RATE=0.1/' \
  ~/apm-lab/apm.env
# Restart both Python processes, then generate 100 requests:
for i in $(seq 1 100); do
  curl -s -o /dev/null \
    'http://127.0.0.1:5050/purchase?mode=normal'
done
```

In Kibana, compare the quantity of detailed sampled traces/spans versus the original 100% configuration. The observed fraction is approximate, not guaranteed to be exactly 10 out of 100. Unsampled transactions may still contribute summary data, but do not contain full span detail.

After the sampling lab, change 0.1 back to 1.0 and restart both apps. Tail-based sampling is a separate APM Server policy and can consume extra local storage and processing; do not enable it on a small VM without checking capacity.

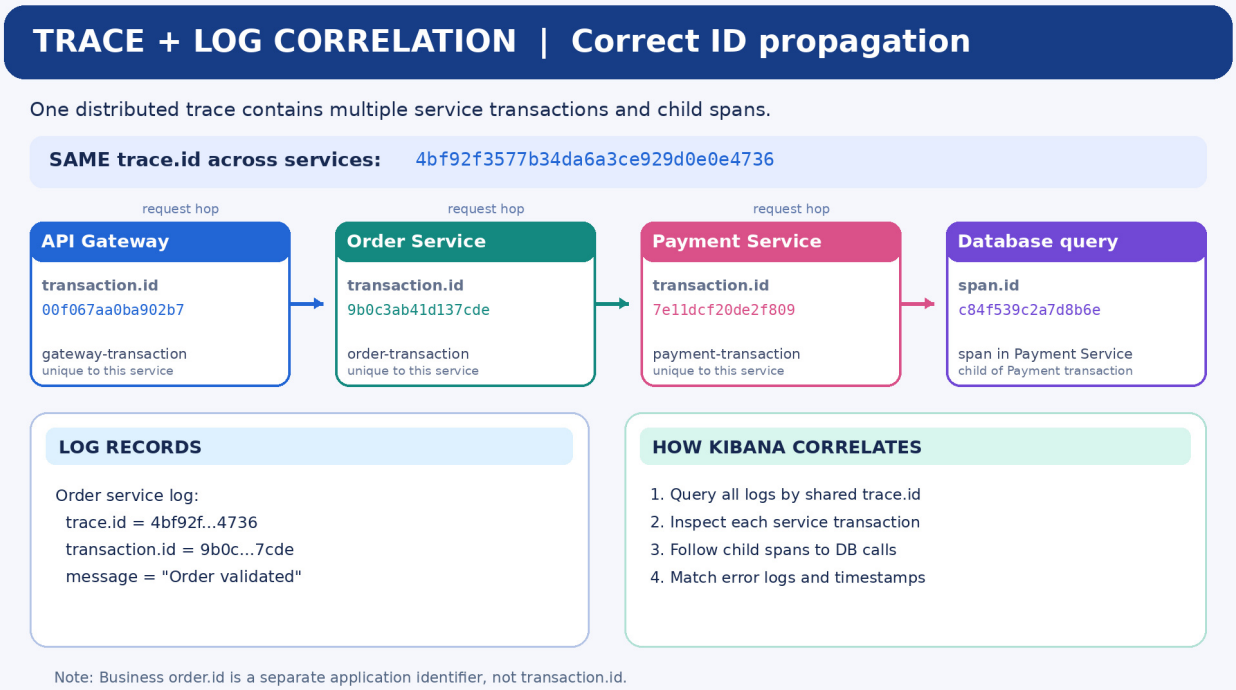
11 | Sampling concepts - visual comparison



Training diagram: head-based selects early; tail-based makes a decision after observing trace events. Example percentages are illustrative.

Question	Correct answer
What does sample rate 0.1 mean?	Approximately 10% of traces are sampled for full detail.
Do all requests disappear?	No. Summary transaction data may still be reported.
Does it change Filebeat logs?	No. Filebeat ingestion has separate filtering and retention.
Why use tail-based policies?	Retain interesting traces such as failed or slow requests.

12 | L5 - Understand trace-to-log correlation



Correct trace ID rules: one trace.id is propagated across services; each transaction/span has its own identifier.

The ECS JSON loggers in orders.py and payments.py are configured to write files beneath /var/log/apm-lab/. When the Python APM agent has tracing context, it makes correlation identifiers available to the logger; ecs-logging serializes them into fields such as trace.id, transaction.id and span.id.

12.1 Verify the files before shipping

```
ls -lh /var/log/apm-lab/
tail -n 2 /var/log/apm-lab/orders.json
tail -n 2 /var/log/apm-lab/payments.json
```

Expected: One JSON record per line containing @timestamp, log.level/message, and correlation identifiers on requests being traced. If identifiers are missing, restart the application after configuring the Elastic APM agent; verify the active sampling rate and the log formatter.

13 | L5 - Ship ECS JSON logs using existing Filebeat

This section is for the app VM that hosts /var/log/apm-lab/*.json. It assumes Filebeat is already installed there, as in your previous ingestion lab. If Filebeat is on a different VM, either run the application on the Filebeat VM or deploy a log collector to the application VM.

13.1 Check current Filebeat and Logstash status

```
sudo systemctl status filebeat --no-pager
sudo filebeat test config -c /etc/filebeat/filebeat.yml
sudo filebeat test output -c /etc/filebeat/filebeat.yml
sudo ss -lntp | grep ':5044'   # run on Logstash VM
```

13.2 Add one filestream input to Filebeat

Edit /etc/filebeat/filebeat.yml. Under the existing filebeat.inputs list append the following item. Do not create another top-level filebeat.inputs or read the same files with duplicate inputs.

```
- type: filestream
  id: apm-lab-ecs-json
  enabled: true
  paths:
    - /var/log/apm-lab/*.json
  parsers:
    - ndjson:
        target: ""
        expand_keys: true
        add_error_key: true
```

Keep the existing output.logstash configuration if it is working. Otherwise it should target the Logstash VM, e.g. output.logstash.hosts ["<LOGSTASH_VM_IP>:5044"], according to your existing pipeline architecture.

13.3 Validate and restart

```
sudo filebeat test config -c /etc/filebeat/filebeat.yml
sudo filebeat test output -c /etc/filebeat/filebeat.yml
sudo systemctl restart filebeat
sudo journalctl -u filebeat -n 60 --no-pager
```

Expected: Configuration OK and Logstash output reachable. If permissions block reads, inspect the Filebeat service user, log directory execute permissions and file read permissions. The APM agent itself does not send these application log files to Elasticsearch.

14 | L5 - Route logs through Logstash

Use your existing working beats input and Elasticsearch output if possible. The goal is only to route this new log source to its own index without disturbing previous Filebeat, CSV or syslog exercises.

14.1 Optional dedicated Logstash output route

If your Logstash pipeline already has beats { port => 5044 }, do not create a second listener on the same port. Add a path-based conditional to the existing output section (and retain any other outputs).

```
if [log][file][path] =~ /\/var\/log\/apm-lab\// {
  elasticsearch {
    hosts => ["https://REPLACE_ES_NODE1_IP:9200"]
    user => "elastic"
    password => "${ES_PWD}"
    ssl_enabled => true
    ssl_certificate_authorities =>
      ["/etc/logstash/certs/http_ca.crt"]
    data_stream => false
    index => "apm-lab-logs-%{+YYYY.MM.dd}"
  }
}
```

Use your existing secured credentials and CA file; the path shown is only an example. Avoid writing passwords directly into pipeline files. If the output already writes to an existing ECS-compatible logs index/data stream, simply reuse it.

14.2 Check service and results

```
sudo /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash -t
sudo systemctl restart logstash
sudo journalctl -u logstash -n 80 --no-pager
```

In Dev Tools confirm the destination index exists:

```
GET _cat/indices/apm-lab-logs-*?v
GET apm-lab-logs-*/_search
{
  "size": 3,
  "sort": [{"@timestamp": "desc"}],
  "_source": ["@timestamp", "service.name", "trace.id",
              "transaction.id", "span.id", "message"]
}
```

> For Logstash deployments using an older Elasticsearch output plugin, the supported TLS parameter names can differ. Adapt only the TLS/output stanza to your installed plugin; keep the validated output from your existing lab if it already works.

15 | L5 - Correlate one failed trace with logs

15.1 Capture the trace ID from APM

- In Kibana APM → order-service, open a failed transaction or distributed trace sample.
- Copy its actual trace.id. Look at the Payment Service segment and inspect the error stack.
- In Kibana Discover select a data view matching apm-lab-logs-*. Set the time filter to Last 30 minutes.
- Search using this KQL, replacing the placeholder with the actual ID:

```
trace.id : "PASTE_ACTUAL_TRACE_ID"
```

Expected: one or more log events written by order-service and payment-service for the same distributed trace. Some spans may not have emitted log events; that is normal.

15.2 Use Dev Tools as a second independent check

```
GET apm-lab-logs-*/_search
{
  "query": { "term": {
    "trace.id": "PASTE_ACTUAL_TRACE_ID"
  }},
  "size": 20,
  "sort": [{"@timestamp": "asc"}],
  "_source": ["@timestamp", "service.name",
              "trace.id", "transaction.id", "message"]
}
```

If your custom index mapped trace.id only as a text field, use the exact keyword subfield if present (such as trace.id.keyword) or install an ECS-compatible index template. See the field mapping in Kibana → Data Views.

15.3 Configure Kibana log sources if needed

If logs are visible in Discover but absent from the APM service Logs tab, check Stack Management → Advanced Settings → observability:logSources and add your custom log index pattern as appropriate for your Kibana version. Changes to log-source settings do not fix missing ingestion.

16 | Correlation exercise and expected findings

Scenario	Trace / APM evidence	Matching log evidence
Normal order	Successful Order and Payment; short duration	Purchase requested; charge completed
Slow order	Payment validate-inventory span ~1.5s	Charging request; charge completed
Failed order	Failure transaction and RuntimeError in APM	Simulated payment failure; order payment failed

16.1 Inspect field identities

For one end-to-end purchase, record the IDs in this worksheet:

Field	Order service	Payment service
trace.id	_____	_____ (same)
transaction.id	_____	_____ (different)
span.id	_____	_____ (per span)
service.name	order-service	payment-service
service.environment	lab	lab

Trace context is propagated in HTTP trace headers by the instrumentation. APM transaction IDs are not your application business order IDs. If you also need a business ID, add a separate field such as order.id to logs; never relabel it as transaction.id.

16.2 Independent learner evidence

- ☐ A trace waterfall linking order-service and payment-service
- ☐ At least two ECS JSON logs with an identical trace.id
- ☐ Two distinct service.name values for the same trace.id
- ☐ A failed transaction and corresponding error log
- ☐ An example unsampled transaction with reduced span detail

17 | Optional - APM alerting and analysis

17.1 Create a basic alert

- Go to Kibana → Observability → Applications → order-service, open Alerts / Rules (the placement varies by version).
- Choose an applicable APM error count/rate or transaction latency rule type. Scope to the order-service and environment lab.
- Configure a low threshold for the demonstration (for example, any error during the selected check window), then generate failing requests and observe rule state.
- A notification connector (webhook, email, etc.) is optional for learning how the rule evaluates. If connectors are disabled, you may still be able to inspect alerts in Kibana without sending a notification, depending on license and role.

17.2 Compare APM with Metricbeat

Metricbeat / infrastructure	APM / application
Host CPU usage	Latency of /purchase
JVM heap (when relevant)	Time spent inside a transaction/span
Disk utilization	Error traces and stack context
Elasticsearch cluster health	Service throughput and failure rate

These tools are complementary: a high CPU metric does not by itself identify which application request or downstream dependency is slow.

18 | Troubleshooting decision table

Symptom	Check / action
No APM integration listener	Fleet Agent Healthy? Policy attached? Check sudo ss -lntp | grep 8200 and elastic-agent logs.
App reaches 8200 but no data	Verify token, Fleet policy output/TLS, agent startup warnings and ES permissions.
Cannot reach APM from app	Check VM IP, private firewall/ufw, 8200 binding, routing.
401 / 403 from intake	Token/API key mismatch; confirm APM authorization configuration.
TLS cert verification fails	Use correct APM Server CA; verify DNS/SAN; do not disable TLS verification.
APM services not in inventory	Set Last 30 minutes; correct environment; ensure Flask debug=False and traffic generated.
Only one service in trace	Check both apps running; HTTP library instrumentation; sampling; service URLs.
No custom span	Confirm sample rate 1.0 and trace waterfall includes Payment /charge.
Filebeat reads 0 log events	Check filestream path, file perms, duplicate input IDs, time and output.
Kibana APM logs tab empty	Verify docs in Discover; check log data source setting; check trace.id field mapping.
Logstash pipeline errors	Check beats 5044 conflict, -t validation, CA and environment secret.
esnode-3 absent	This is cluster recovery work, not an APM agent issue; fix before APM lab.

Diagnostics command sheet

```
# APM host (Kibana VM or designated APM VM)
sudo systemctl status elastic-agent --no-pager
sudo journalctl -u elastic-agent -n 100 --no-pager
sudo ss -lntp | grep -E ':8200|:8220'
# App host
curl -iv --max-time 5 http://REPLACE_APM_VM_IP:8200/
python -m pip show elastic-apm ecs-logging
# Filebeat host
sudo journalctl -u filebeat -n 100 --no-pager
# Logstash host
sudo journalctl -u logstash -n 100 --no-pager
```

19 | Lab completion and validation checklist

- ☐ All three intended Elasticsearch nodes are members of the cluster.
- ☐ Kibana is reachable and Fleet Agent assigned to APM policy is Healthy.
- ☐ APM intake listens on 8200 and is reachable from the app VM.
- ☐ Order and Payment Flask processes run on ports 5050 and 5051.
- ☐ Normal /purchase returns HTTP 200; fail returns HTTP 500.
- ☐ Service Inventory lists order-service and payment-service in lab.
- ☐ A distributed trace contains both services with the same trace.id.
- ☐ Slow request contains a visibly long payment custom span.
- ☐ At 10% sampling, detailed trace spans are reduced vs 100%.
- ☐ ECS JSON log files exist and are being shipped by a log collector.
- ☐ Kibana Discover can find logs with same trace.id as selected APM trace.
- ☐ Screenshots collected for service list, trace waterfall, error and logs.

Suggested completion rubric

Area	Points
APM policy/ingestion verified	20
Two services with distributed traces	25
Slow/error analysis	20
Sampling and naming demonstration	15
Log correlation + Dev Tools evidence	20
TOTAL	100

Safe lab cleanup

Stop the two Flask development processes with Ctrl+C. Keep the APM integration if you will continue working with it. Remove or disable only the dedicated Filebeat input if you no longer want these logs collected. Do not uninstall Elastic Agent, Fleet Server, Kibana or Elasticsearch as part of this cleanup.

20 | Reference links and notes

Elastic official documentation reviewed for this lab:

Fleet-managed APM Server - https://www.elastic.co/docs/solutions/observability/apm/apm-server/fleet-managed

Elastic APM Python agent configuration - https://www.elastic.co/docs/reference/apm/agents/python/configuration

Flask support - https://www.elastic.co/docs/reference/apm/agents/python/flask-support

APM transaction sampling - https://www.elastic.co/docs/solutions/observability/apm/transaction-sampling

Python agent log correlation - https://www.elastic.co/docs/reference/apm/agents/python/logs

ECS logging Python - https://www.elastic.co/docs/reference/ecs/logging/python/installation

Filebeat filestream input - https://www.elastic.co/docs/reference/beats/filebeat/filebeat-input-filestream

APM data streams - https://www.elastic.co/docs/solutions/observability/apm/data-streams

APM application logs - https://www.elastic.co/docs/solutions/observability/apm/logs

APM agent TLS communication - https://www.elastic.co/docs/solutions/observability/apm/apm-agent-tls-communication

> Version awareness: Commands in this manual target the current Fleet-managed Elastic APM model and supported Python APM agent. Older 8.x releases may use different Kibana navigation or Logstash TLS parameter names. Keep your existing working Elasticsearch TLS/output configuration when integrating.

File package contents

File	Purpose
orders.py	Flask order entry point with outbound HTTP tracing and ECS logs
payments.py	Flask payment dependency, custom span, slow and failure modes
apm.env.template	Local APM intake URL, token and sample rate
filebeat_apm_input.yml	Filestream JSON log input fragment
logstash_apm_pipeline.conf	Optional example Beats input and Elasticsearch output

This manual uses simulated transactions and errors solely for teaching. No real card details, production secrets or actual screenshots are included.